

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ВГРАДЕНО ХРАНИЛИЩЕ ЗА RDF ТРИПЛЕТИ
С МЕХАНИЗЪМ ЗА ИЗПЪЛНЕНИЕ НА
ЗАЯВКИ НА SPARQL 1.1**

КУРСОВ ПРОЕКТ

по дисциплина „Семантичен уеб“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Доц. д-р инж. Аделина Алексиева-Петрова

София, 2027

СЪДЪРЖАНИЕ

УВОД	1
I. ПРЕДМЕТНА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
1.1. Информационен модел на семантичния уеб	3
1.2. Синтаксиси за обмен на RDF	3
1.3. Семантика, онтологии и правила	3
1.4. Език за заявки и подходи за индексирание	4
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	5
2.1. Целеви език и среда за изграждане	5
2.2. Кодирание на термините	5
2.3. Набор от индекси	5
2.4. Носител на индексите	6
2.5. Вид на синтактичните анализатори	6
2.6. Стратегия на планировчика	7
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	8
3.1. Слоеви и отговорности	8
3.2. Речник на термините	8
3.3. Формат на индексите	9
3.4. Алгебрично дърво и план за изпълнение	9
3.5. Слой за извеждане по RDFS	9
3.6. Поведение при грешка	9
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	11
4.1. Обща структура на изходния текст	11
4.2. Йерархия на типовете	11
4.3. Синтактични анализатори	11
4.4. Механизъм за изпълнение	12
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	13
5.1. Какво се проверява	13

5.2. Коректност	13
5.3. Еталонни реализации и набори от данни	13
5.4. Измервани величини	14
5.5. Условия за валидност на измерването	14
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	15
6.1. Коректност спрямо тестовите набори	15
6.2. Еднократна цена: зареждане и обем	15
6.3. Повтаряща се цена: изпълнение на заявки	15
6.4. Цена на извеждането по RDFS	16
6.5. Анализ	16
ЗАКЛЮЧЕНИЕ	17
ЛИТЕРАТУРА	18
ПРИЛОЖЕНИЯ	19

УВОД

Семантичният уеб описва информацията във вид, който машина може да свърже, вместо само да покаже. Основната единица на този запис е *триплетът*: подлог, сказуемо и допълнение, всяко от които е ресурс с идентификатор или литерална стойност. Моделът RDF задава абстрактния синтаксис на този запис [1], а езикът SPARQL задава как над множество триплети се формулира заявка [2]. Двете спецификации са нормативни и публични, но между тях и работеща програма стои инженерна задача, която спецификациите съзнателно не решават: как триплетите се съхраняват, индексират и обхождат така, че заявката да завърши за приемливо време.

Съществуващите реализации решават тази задача, но повечето от тях са съвърши. Те предполагат отделен процес, мрежов протокол и администриране. Има случаи, в които това е излишно: настолно приложение, вграден инструмент за анализ, конвейер за обработка на данни, който трябва да зададе няколко заявки над локален набор от триплети и да продължи. За такъв случай е подходящо *вградено* хранилище, тоест библиотека, която се свързва към приложението и работи в неговия процес и адресно пространство.

Целта на курсовия проект е да се проектира и реализира вградено хранилище за RDF триплети с механизъм за изпълнение на заявки на SPARQL 1.1, и да се измери поведението му спрямо утвърдени реализации върху едни и същи данни и заявки.

За постигането на целта са формулирани следните **задачи**:

1. преглед на информационния модел на семантичния уеб, на синтаксисите за обмен на RDF и на семантиката на SPARQL, заедно с публикуваните подходи за индексване на триплети;
2. анализ на възможните проектни решения по всяко от отворените места, тоест начин на кодиране на термините, набор от индекси, вид на синтактичния анализатор и стратегия на планировчика, с обосновка на направения избор;
3. проектиране на слоеста архитектура, в която синтаксисът, съхранението, анализът на заявки, планирането и изпълнението са отделени един от друг;
4. програмна реализация на архитектурата на C++20 с изграждане чрез CMake;
5. изграждане на методика за проверка на коректността спрямо публичните тестови набори на W3C и за измерване на времето за изпълнение спрямо еталонни реализации;

6. провеждане на измерванията, анализ на получените резултати и извеждане на заключения за приложимостта на избраните решения.

Обхватът на проекта покрива четене и записване на Turtle и N-Triples, съхранение с речниково кодиране и пермутирани индекси, синтактичен анализ на SPARQL 1.1 до алгебрично дърво, планиране на реда на съединенията и изпълнение над индексите. Правилата за извеждане по RDFS се разглеждат като отделен, незадължителен слой. Извън обхвата остават протоколът SPARQL през HTTP, езикът SPARQL Update, разпределеното изпълнение и пълното разсъждаване по OWL 2; последното се разглежда само описателно, доколкото дисциплината го включва.

Предходна работа на автора. Дипломната работа на автора за ОКС „Бакалавър“ е библиотека на C++ за обектно релационно съответствие, изградена върху изчисления по време на компилация. Общото с настоящия проект е проектирането на език за заявки и превръщането му в изпълним план, което тук се пренася върху друг модел на данните. Изходният текст на предходната работа не се използва. **[TODO: точно заглавие и година на дипломната работа за ОКС „Бакалавър“, за цитиране]**

Структура на изложението. Раздел I разглежда предметната област и литературата. Раздел II излага разгледаните алтернативи и обосновава избора. Раздел III представя проектирането, а Раздел IV програмната реализация. Раздел V описва методиката за проверка и измерване, Раздел VI получените резултати.

I. ПРЕДМЕТНА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

1.1. Информационен модел на семантичния уеб

Класическият уеб свързва документи. Връзката между тях е хипервръзка без тип: тя казва, че два документа са свързани, но не и как. Семантичният уеб добавя точно този липсващ пласт, като описва не документите, а ресурсите, и дава на всяка връзка собствен идентификатор и значение. Единицата на записа е триплет от подлог, сказуемо и допълнение. Множество триплети образува насочен етикетирани граф, в който допълнението на един триплет може да бъде подлог на друг, и така отделно съставени описания се свързват без предварително договаряне на обща схема.

Абстрактният синтаксис на този модел е дефиниран в спецификацията RDF 1.1, която въвежда трите вида термини, тоест IRI, литерал и празен възел, и определя графа и множеството от графи [1]. Спецификацията умишлено разделя абстрактния модел от конкретните му записи. Това разделение е важно за настоящия проект: то позволява синтактичните анализатори да се разглеждат като сменяеми входове към един и същ вътрешен модел.

1.2. Синтаксиси за обмен на RDF

Обменът на RDF между системи става чрез конкретен синтаксис. Turtle е компактен запис, четим от човек, с префикси, съкратени форми за списъци и вложени описания на празни възли [3]. N-Triples е подмножество на Turtle, в което всеки ред съдържа точно един пълен триплет без съкращения [4]. Заради тази простота N-Triples е удобен за поточна обработка и за тестови набори, докато Turtle е предпочитан за ръчно писани описания. Проектът поддържа и двата, като N-Triples се третира като частен случай в същия синтактичен анализатор.

1.3. Семантика, онтологии и правила

Синтаксисът определя кои записи са допустими, но не и какво следва от тях. Семантиката на RDF задава кога един граф следва от друг, тоест отношението на извеждане [5]. RDF Schema надгражда с речник за класове, свойства, области и обхвати, и с това позволява прости правила от вида „ако ресурс принадлежи на подклас, той

принадлежи и на надкласа“ [6]. Езикът OWL 2 разширява изразителната сила значително, като добавя ограничения върху свойствата, кардиналности, дизюнктни класове и други конструкции, и с това променя изчислителната сложност на извеждането [7]. Настоящият проект реализира извеждане по RDFS като отделен слой и разглежда OWL 2 само описателно.

1.4. Език за заявки и подходи за индексирание

SPARQL 1.1 е нормативният език за заявки над RDF. Той дефинира базови графови шаблони, незадължителни части, обединения, филтри, агрегация, подзаявки и транслативни пътища по свойства, а също и алгебра, към която текстът на заявката се превежда преди изпълнение [2]. Тази алгебра е удобната граница между синтактичния анализатор и механизма за изпълнение и е приета за такава и в този проект.

Литературата за съхранение на триплети се съсредоточава върху един въпрос: базовият графов шаблон изисква многократно търсене по различни комбинации от свързани и свободни позиции, а един ред на сортиране обслужва добре само част от тях. Hexastore предлага пълния набор от шест пермутации на подлог, сказуемо и допълнение, с което всяка комбинация се обслужва от подходящо подреден индекс, срещу цената на шесткратно повторение на данните [8]. RDF-3X показва, че при речниково кодиране на термините и компресирани индекси този подход е практически приложим, и добавя планировчик, който избира реда на съединенията по статистики върху самите индекси [9]. Двете публикации формират основата на проектното решение в Раздел III.

[TODO: преглед на поне два по-нови източника за изпълнение на SPARQL, за да не е прегледът съсредоточен само в периода 2008 до 2010]

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Целеви език и среда за изграждане

Разгледани са три възможности: C++20, Rust и Java. Java отпада заради изискването хранилището да бъде вградено в приложение на друг език без отделна среда за изпълнение. Rust покрива изискването и предлага по-строги гаранции за паметта, но свързването му към съществуващ код на C++ минава през допълнителен слой. Избран е C++20, защото целевото приложение на такава библиотека най-често е на C++, защото езикът дава пряк контрол върху разположението на данните в паметта, което е определящо за индексите, и защото авторът има опит в него. Изграждането е с CMake, тъй като това е фактическият стандарт за преносими проекти на C++.

2.2. Кодиране на термините

Триплетът може да съхранява самите низове на термините или числови идентификатори, получени от речник. Прякото съхранение на низове е по-просто, но повтаря един и същ IRI толкова пъти, колкото триплета го използват, а сравненията при съединяване стават низови. Речниковото кодиране изисква два допълнителни справочника, от термин към идентификатор и обратно, но свежда триплета до три числа с фиксиран размер. Това прави индексите плътни, сравненията евтини, а съединенията сводими до сливане на сортирани редици. Избрано е речниково кодиране, следвайки [9].

2.3. Набор от индекси

Пълният набор от шест пермутации обслужва всяка комбинация от свързани позиции с подходящо подреден индекс [8], но умножава обема шест пъти. Един единствен индекс по подлог е компактен, но при шаблон със свободен подлог води до пълно обхождане. Избрани са три пермутации, а именно подлог казуемо допълнение, казуемо допълнение подлог и допълнение подлог казуемо. Всяка от осемте комбинации от свързани и свободни позиции се обслужва от префикс на поне един от трите индекса, което дава покритие без шесткратното повторение. Обосновката е обобщена в (Табл. 1).

2.4. Носител на индексите

Възможностите са структури изцяло в паметта, вграден ключ и стойност склад като вариант на дърво с логаритмично сливане, и собствени файлове, отразени в паметта. Структурите в паметта са най-бързи, но задължават целият набор да се събира в оперативната памет. Вграден външен склад решава това, но внася зависимост, чийто модел на съхранение не съвпада с подредените редици, които съединяването изисква. Избрани са собствени файлове, отразени в паметта: подредбата на записите остава под контрола на проекта, операционната система поема кеширането, а отварянето на съществуващо хранилище не изисква повторно изграждане на индексите.

2.5. Вид на синтактичните анализатори

Граматиките на Turtle и на SPARQL 1.1 са публикувани в спецификациите и могат да бъдат подадени на генератор на анализатори. Генераторът пести писане и намалява риска от разминаване с граматиката, но добавя зависимост, стъпка в изграждането и генериран изходен текст, чиито съобщения за грешка са трудни за привеждане към полезен вид. Двете граматики са проектирани така, че да се разпознават с ограничен предварителен поглед, което ги прави удобни за анализатор с рекурсивно спускане, писан на ръка. Избран е вторият вариант, като рискът от разминаване с граматиката се покрива от тестовите набори на W3C [10].

Таблица 1. Обобщение на разгледаните алтернативи и на направения избор

Решение	Разгледани варианти	Избор	Водещ довод
Език	C++20, Rust, Java	C++20	вграждане без отделна среда за изпълнение
Термини	низове в триплета, речниково кодиране	речниково кодиране	фиксиран размер, евтини сравнения
Индекси	един, три, шест пермутации	три пермутации	покритие на всички шаблони без шесткратен обем
Носител	памет, външен склад, отразени файлове	отразени файлове	контрол върху подредбата, без нова зависимост
Анализатори	генератор, рекурсивно спускане	рекурсивно спускане	по-добри съобщения, без стъпка в изграждането

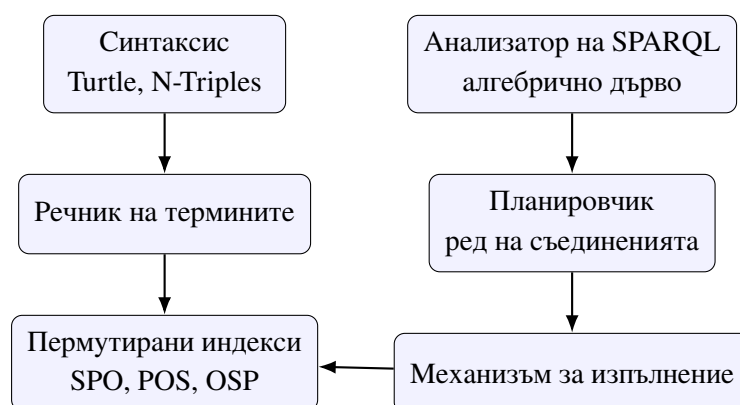
2.6. Стратегия на планировчика

Редът на съединенията при базов графов шаблон определя размера на междинните резултати и с това времето за изпълнение. Разгледани са два подхода: фиксирано правило, което подрежда шаблоните по брой свързани позиции, и избор по оценка, основан на статистики от самите индекси. Първият е тривиален за реализация и няма цена по време на планиране, но е чувствителен към реда на изписване на заявката. Вторият изисква поддържане на статистики, но остава устойчив. Избран е вторият, като статистиките се извличат от дължините на префиксите в индексите. **[TODO: решение дали статистиките се поддържат постъпково при вмъкване или се преизчисляват при отваряне на хранилището]**

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

3.1. Слоеве и отговорности

Системата е разделена на шест слоя, като всеки от тях зависи само от слоевете под себе си и общува с тях през тесен договор. Слойът на синтаксиса чете и записва Turtle и N-Triples и превръща текста в поток от триплети над термини. Речникът на термините съпоставя всеки термин с числов идентификатор и обратно. Слойът на индексите поддържа трите пермутирани подредби на триплетите. Анализаторът на SPARQL превръща текста на заявката в алгебрично дърво. Планировчикът преобразува това дърво в план за изпълнение, като избира реда на съединенията. Механизмът за изпълнение обхожда плана и произвежда решенията. Разположението на слоевете и посоката на данните са показани на (Фиг. 1).



Фигура 1. Слоеве на системата. Лявата колона е пътят на данните при зареждане, дясната е пътят на заявката, а стрелката между тях е единственият достъп на изпълнението до индексите

3.2. Речник на термините

Речникът поддържа две посоки. Посоката от термин към идентификатор се използва при зареждане и при превръщане на константите в заявката, а обратната само при извеждане на резултата. Идентификаторите носят в себе си вида на термина, тоест IRI, литерал или празен възел, така че механизмът за изпълнение да може да прилага филтри без да чете самия термин. Празните възли са локални за документа, от който идват, и се преименуват при зареждане, за да не се сблъскат имената от два различни документа. **[TODO: ширина на идентификатора и разпределение на битовите между вид и пореден номер]**

3.3. Формат на индексите

Всеки индекс е редица от триплети идентификатори, подредена лексикографски по съответната пермутация, и записана в отделен файл, който се отразява в паметта при отваряне. Търсенето по шаблон се свежда до двоично търсене на границите на префикса, съответстващ на свързаните позиции, последвано от последователно четене. Тази форма е избрана заради две свойства: последователното четене е предвидимо за подсистемата за виртуална памет, а подредеността позволява съединяването да бъде сливане на сортирани редици, без изграждане на хеш таблица. **[TODO: решение за компресията на индексите и дали тя се въвежда в обхвата на проекта]**

3.4. Алгебрично дърво и план за изпълнение

Анализаторът на SPARQL произвежда дърво, чиито възли съответстват на операторите от алгебрата на спецификацията, а именно базов графов шаблон, съединение, незадължително съединение, обединение, филтър, проекция, подреждане и ограничение на броя резултати [2]. Планировчикът работи само върху това дърво и не познава синтаксиса, което позволява двете части да се тестват поотделно. Планът за изпълнение е същото дърво с фиксиран ред на съединенията и с избран индекс за всеки базов шаблон.

3.5. Слой за извеждане по RDFS

Извеждането по RDFS се проектира като незадължителен слой, който може да работи в два режима. При материализация правилата се прилагат след зареждане и получените триплети се вмъкват в индексите. При извеждане по време на заявка правилата се прилагат върху резултата от обхождането. Първият режим утежнява зареждането и обема, вторият утежнява всяка заявка. Проектът предвижда и двата, за да може изборът да бъде измерен, а не предположен. **[TODO: списък на подмножеството от правила по RDFS, което се реализира]**

3.6. Поведение при грешка

Грешките се разделят на три вида: синтактична грешка във входния документ или в заявката, нарушено съответствие в самото хранилище и изчерпан ресурс. Първият вид се съобщава с позиция в текста, ред и колона, и не оставя частично заредени данни.

Вторият вид спира операцията, защото продължаването върху повреден индекс дава тихо грешен отговор, което е по-лошо от отказ. Третият вид се съобщава на извикващия код без изключение, тъй като библиотеката трябва да може да работи и при изключен механизъм за изключения.

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

4.1. Обща структура на изходния текст

Проектът е организиран в три дървета. В `include/` стоят публичните заглавни файлове, тоест единственото, което вижда приложението, използващо библиотеката. В `src/` стои реализацията, разделена на подпапки по слоевете от Раздел III. В `tests/` стоят тестовете, писани с GoogleTest. Разделянето на публично от вътрешно е съзнателно: то позволява представянето на индексите да се промени, без да се променя нито един ред в кода на потребителя на библиотеката.

Изграждането е с CMake. Библиотеката се изгражда като статична по подразбиране, тъй като целевият случай на употреба е вграждане. Тестовете са отделна цел, която може да бъде изключена, за да не изисква изтегляне на GoogleTest при вграждане в чужд проект.

4.2. Йерархия на типовете

Ключовите типове следват слоевете. Тип за термин, който представя IRI, литерал или празен възел. Тип за идентификатор на термин, който е обвивка над цяло число и не допуска неявно превръщане, за да не се разменят по грешка идентификатори от различни речници. Тип за триплет от три идентификатора. Речник, който предоставя двете посоки на съпоставянето. Индекс, който предоставя обхождане по префикс. Възли на алгебричното дърво, обединени в един вариант, вместо в йерархия с виртуални методи, защото над дървото се извършват няколко различни обхождания, а не едно.

[TODO: диаграма на класовете и на зависимостите между модулите]

4.3. Синтактични анализатори

Двата анализатора споделят общ четец на знаци, който води ред и колона и предоставя предварителен поглед с един знак. Върху него анализаторът на Turtle разпознава префиксните обявявания, съкратените форми и вложените описания, а анализаторът на SPARQL разпознава конструкциите на езика. Общият четец е причината съобщенията за грешка да имат еднакъв вид и в двата случая.

[TODO: коментар на изходния текст на анализатора на SPARQL с листинг на представителна функция, след като бъде написан]

4.4. Механизъм за изпълнение

Изпълнението е организирано като верига от итератори, всеки от които произвежда следващото решение при поискване. Този модел е избран, за да не се материализират междинните резултати изцяло и за да може ограничението на броя резултати да спре работата рано. Съединяването на два подредени входа се извършва чрез сливане, а при несъвместими подредби се вмъква стъпка за подреждане.

[TODO: описание на реализираните оператори и на тези, които остават извън обхвата]

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

5.1. Какво се проверява

Проверката е разделена на две независими части, защото отговарят на различни въпроси. Проверката за коректност пита дали системата дава отговора, който спецификацията изисква. Измерването на производителността пита колко струва този отговор. Втората част няма смисъл преди първата да е удовлетворена: бърз, но грешен отговор не е резултат.

5.2. Коректност

Коректността се проверява спрямо публичните тестови набори на W3C. За синтаксиса се използват положителните и отрицателните случаи за Turtle и N-Triples, тоест документи, които трябва да бъдат приети, и документи, които трябва да бъдат отхвърлени. За заявките се използва тестовият набор на SPARQL 1.1, в който всеки случай съдържа входен граф, текст на заявка и очакван резултат [10]. Резултатът от тази част е брой преминали, брой непреминали и брой пропуснати случаи по категории, заедно с изричен списък на конструкциите, които остават извън обхвата.

[TODO: точна версия на тестовия набор и начин на записване на пропуснатите случаи]

5.3. Еталонни реализации и набори от данни

Производителността се сравнява с две утвърдени реализации: Apache Jena [11] и RDFLib [12]. Двете са избрани, защото са широко използвани и защото представят два различни компромиса, единият върху среда за изпълнение с управление на паметта, другият върху интерпретиран език. Сравнението се извършва върху едни и същи входни файлове и едни и същи текстове на заявки.

[TODO: избор на наборите от данни, техният размер и произход] [TODO: списък на заявките, които образуват работното натоварване, с кратко описание какво натоварва всяка от тях]

5.4. Измервани величини

Измерват се четири величини: време за зареждане на набора от данни, обем на получените индекси на диска, време за изпълнение на всяка заявка и върхова потребена памет по време на изпълнението. Първите две характеризират еднократната цена, вторите две повтарящата се.

5.5. Условия за валидност на измерването

Едно измерване се приема за валидно само ако са изпълнени следните условия. Всяка конфигурация се изпълнява многократно и се докладва медианата, а не единично измерване. Първите изпълнения се отхвърлят, за да не се отчита загряването на кешовете на файловата система. Всички реализации четат едни и същи файлове от един и същ носител. Проверява се, че всяка реализация е върнала един и същ брой решения преди времето ѝ да бъде записано, тъй като реализация, която връща по-малко решения, е по-бърза без това да значи нещо.

[TODO: брой повторения и брой отхвърляни загряващи изпълнения] [TODO: описание на машината, върху която се провеждат измерванията]

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

6.1. Коректност спрямо тестовите набори

Резултатът от проверката за коректност се представя в (Табл. 2). Всеки непреминал случай се разглежда поотделно и се отнася към една от две причини: липсваща конструкция, която е извън обявения обхват, или грешка в реализацията. Разграничението е съществено, защото първата причина е проектно решение, а втората е дефект.

Таблица 2. Резултати от тестовите набори на W3C

Тестов набор	Общо	Преминали	Непреминали	Пропуснати
Turtle	[TODO: n]	[TODO: n]	[TODO: n]	[TODO: n]
N-Triples	[TODO: n]	[TODO: n]	[TODO: n]	[TODO: n]
SPARQL 1.1	[TODO: n]	[TODO: n]	[TODO: n]	[TODO: n]

[TODO: списък на непреминалите случаи с причината за всеки от тях]

6.2. Еднократна цена: зареждане и обем

Времето за зареждане и обемът на получените индекси се представят в (Табл. 3). Величините се отчитат за една и съща входна редица от триплети и при еднакви условия по методиката от Раздел V.

Таблица 3. Време за зареждане, обем на индексите и върхова потребена памет

Реализация	Зареждане, s	Обем, MB	Памет, MB
Trident	[TODO: изм.]	[TODO: изм.]	[TODO: изм.]
Jena	[TODO: изм.]	[TODO: изм.]	[TODO: изм.]
RDFLib	[TODO: изм.]	[TODO: изм.]	[TODO: изм.]

6.3. Повтаряща се цена: изпълнение на заявки

Времената за изпълнение по заявки се представят в (Табл. 4). Колоната с броя решения не е допълнителна информация, а условие за валидност: ред, в който броевете не съвпадат, не се тълкува като сравнение на скорост.

Таблица 4. Медианно време за изпълнение по заявки

Заявка	Trident, ms	Jena, ms	RDFLib, ms	Решения
[TODO: заявка]	[TODO: изм.]	[TODO: изм.]	[TODO: изм.]	[TODO: n]

[TODO: редовете на таблицата се добавят, след като се определи наборът от заявки в Раздел V; всяка заявка получава ред]

6.4. Цена на извеждането по RDFS

Двата режима на слоя за извеждане се сравняват помежду си и с изпълнение без извеждане. Очакваният вид на резултата е компромис между еднократна и повтаряща се цена, но посоката и големината му се записват едва след измерване.

[TODO: таблица за сравнение на двата режима на извеждане]

6.5. Анализ

[TODO: анализ на получените резултати: къде избраните проектни решения се потвърждават, къде не, и коя от тях е причината за наблюдаваната разлика]

ЗАКЛЮЧЕНИЕ

Проектът разглежда една задача от предметната област на семантичния уеб, а именно съхранението на RDF триплети и изпълнението на заявки над тях, и я решава във вид, подходящ за вграждане в приложение. Избраната архитектура разделя синтаксиса, съхранението, анализа на заявката, планирането и изпълнението на слоеве с тесни граници помежду им, което позволява всеки от тях да бъде проверяван поотделно.

Предимства на избраното решение. Речниковото кодиране и подредените пермутирани индекси свеждат съединяването до сливане на сортирани редици и премахват низовите сравнения от горещия път. Отразяването на индексите в паметта пренася кеширането към операционната система и позволява отваряне на съществуващо хранилище без повторно изграждане. Липсата на външни зависимости извън средството за изграждане и тестовата рамка прави вграждането просто.

Недостатъци и ограничения. Три пермутации вместо шест означават, че част от шаблоните се обслужват от индекс, който не е подреден по най-удобния за тях начин. Собственият формат на индексите е отговорност на проекта, включително съвместимостта му при промяна. Обхватът не покрива протокола на SPARQL през HTTP, езика за промяна на данни и разсъждаването по OWL 2.

[TODO: инженерна трактовка на измерените резултати: какво показват таблиците от Раздел VI и какво следва от тях за приложимостта на решението]

Предложения за по-нататъшна работа. Естествените продължения са три: компресия на индексите, която да намали обема без да разруши възможността за двоично търсене по префикс; разширяване на слоя за извеждане отвъд подмножеството по RDFS; и поддръжка на именувани графи заедно със съответното разширяване на индексите до четворки.

ЛИТЕРАТУРА

- [1] R. Cyganiak, D. Wood, and M. Lanthaler, “RDF 1.1 concepts and abstract syntax.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/rdf11-concepts/>.
- [2] S. Harris and A. Seaborne, “SPARQL 1.1 query language.” W3C Recommendation, Mar. 2013. <https://www.w3.org/TR/sparql11-query/>.
- [3] E. Prud’hommeaux and G. Carothers, “RDF 1.1 Turtle: Terse RDF triple language.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/turtle/>.
- [4] G. Carothers and A. Seaborne, “RDF 1.1 N-Triples: A line-based syntax for an RDF graph.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/n-triples/>.
- [5] P. J. Hayes and P. F. Patel-Schneider, “RDF 1.1 semantics.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/rdf11-mt/>.
- [6] D. Brickley and R. V. Guha, “RDF Schema 1.1.” W3C Recommendation, Feb. 2014. <https://www.w3.org/TR/rdf-schema/>.
- [7] W3C OWL Working Group, “OWL 2 Web Ontology Language document overview (second edition).” W3C Recommendation, Dec. 2012. <https://www.w3.org/TR/owl2-overview/>.
- [8] C. Weiss, P. Karras, and A. Bernstein, “Hexastore: sextuple indexing for semantic web data management,” *Proceedings of the VLDB Endowment*, vol. 1, no. 1, pp. 1008–1019, 2008.
- [9] T. Neumann and G. Weikum, “The RDF-3X engine for scalable management of RDF data,” *The VLDB Journal*, vol. 19, no. 1, pp. 91–113, 2010.
- [10] W3C SPARQL Working Group, “SPARQL 1.1 test suite,” 2013. <https://www.w3.org/2009/sparql/docs/tests/>.
- [11] The Apache Software Foundation, “Apache Jena documentation,” **[TODO: година на използваната версия]**. Версия: **[TODO: версия на Jena, използвана при измерванията]**. <https://jena.apache.org/documentation/>.
- [12] RDFLib Team, “RDFLib documentation,” **[TODO: година на използваната версия]**. Версия: **[TODO: версия на RDFLib, използвана при измерванията]**. <https://rdflib.readthedocs.io/>.

ПРИЛОЖЕНИЯ

Приложение А. Формат на файловете с индекси

[TODO: двоичният формат на един индексен файл: заглавен блок, ред на записите, подравняване и признак за версия]

Приложение Б. Формат на речника на термините

[TODO: представяне на двете посоки на съпоставянето и начин на кодиране на вида на термина в идентификатора]

Приложение В. Граматика на поддържаното подмножество на SPARQL

[TODO: изброяване на продукциите от граматиката в спецификацията, които се разпознават, и на онези, които се отхвърлят с изрична грешка]

Приложение Г. Изходен текст на избрани модули

[TODO: листинги на анализатора на заявки и на оператора за съединяване чрез сливане, след като бъдат написани]

Приложение Д. Заявки, използвани при измерванията

[TODO: пълният текст на всяка заявка от работното натоварване по Раздел V]

Приложение Е. Указания за изграждане и стартиране

[TODO: командите за изграждане и за пускане на тестовете, сверени с README на хранилището с изходния текст]